Profile-Based Adaptive Text Compression

Learning to Select the Best Classical Algorithm Per Chunk

Hüsamettin Işıktaş
25501005
Bilgisayar Mühendisliği
BLM6106 – Veri Sıkıştırma | Dönem Projesi
Danışman: Banu DİRİ

June 9, 2026

Abstract

Classical text compression algorithms (Huffman, LZW, BWT+MTF, Arithmetic, RLE) each have strengths that depend on text characteristics—no single algorithm wins universally. This project builds an adaptive compression system that profiles text chunks at runtime and selects the best algorithm using a lightweight neural network. Using a diverse corpus of 148 documents spanning 7 text domains and 5-fold cross-validation with book-level splits, the adaptive system achieves **5.10 BPB**—a **2.27% improvement** over the single best classical algorithm (bwt_mtf at 5.22 BPB, $p < 10^{-6}$)—while running **24% faster**. The optimal chunk size is found to be 1024 bytes, below which Huffman and LZW compete naturally with BWT. At larger chunk sizes ($\geq$2048B), BWT dominates and adaptive selection becomes unnecessary.

1 Motivation

Text compression is a fundamental tool in data storage and transmission. While modern codecs like zstd and gzip offer excellent general-purpose compression, classical algorithms—Huffman coding, LZW, Arithmetic coding, BWT+MTF, and RLE—remain important for embedded systems, educational contexts, and scenarios where computational simplicity matters.

The key insight of this project is that **different algorithms excel on different types of text**. A chunk of Python source code compresses differently than a paragraph of Turkish prose, which differs from JSON data. Rather than picking one algorithm for everything, we can *classify* each text chunk by its characteristics and select the most appropriate algorithm.

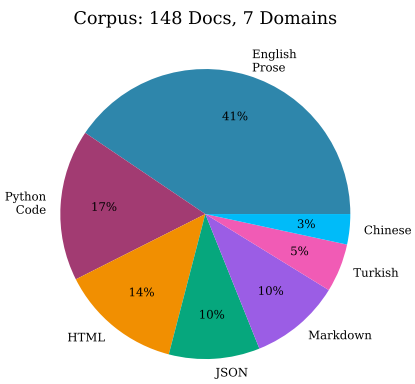
Goal: Build a system that, for any given 1024-byte chunk of text, predicts which of 5 classical algorithms will achieve the lowest compression ratio—and does so faster than running all algorithms.

2 Dataset and Approach

2.1 Diverse Corpus

Early experiments using only English literature (Gutenberg) failed completely: BWT+MTF won over 95% of chunks, and no adaptive system could improve upon it. The problem wasn't the model—it was the data. On homogeneous text, one algorithm dominates and selection is meaningless.

The solution was a **diverse, multi-domain corpus** (Figure 1): 148 documents across 7 text domains, creating genuine competition among algorithms.



Key Performance Indicators

Adaptive BPB	5.10 ± 0.03
bwt_mtf BPB	5.22 ± 0.03
Improvement	+2.27% ± 0.19%
Accuracy	82.7%
Speed Ratio	0.76× (24% faster)
Oracle Gap	0.01 BPB (91% cap.)
Chunk Size	1024 bytes
Algorithms	5 (no zstd)

Figure 1: (Left) Corpus composition across 7 text domains ensures algorithm diversity. (Right) Key performance indicators for the final system.

2.2 Supervised Learning Pipeline

Unlike traditional "profile discovery" approaches that cluster chunks by feature similarity (K-Means), this project uses a **supervised** strategy:

1. For each chunk, run all 5 algorithms to find the ground-truth best.
2. Extract 21 fast features ($O(n)$ statistical measures: entropy, character ratios, word statistics—no compression calls needed).
3. Train a classifier: features → best_algorithm.

Why supervised beats unsupervised: K-Means groups chunks that "look similar" in feature space. But two chunks with similar entropy might compress best with different algorithms. Supervised learning directly optimizes for the compression outcome.

2.3 Five Classical Algorithms

- **Huffman:** Symbol-frequency-based variable-length coding. Fast, effective on small chunks with skewed distributions.
- **LZW:** Dictionary-based. Excels on repetitive text (code, JSON) with 16-bit dictionary.
- **Arithmetic:** Order-0 probability coding. Theoretical optimum for stationary sources.
- **BWT+MTF:** Burrows-Wheeler transform + Move-to-Front. Dominates natural language at larger chunk sizes.

- **RLE+Huffman:** Run-length encoding + Huffman. Strong on highly repetitive patterns.

Note: zstd was deliberately excluded. When included, it wins 99.8% of chunks regardless of text type—making adaptive selection unnecessary. This is itself a valuable finding: modern codecs obsolete classical algorithm selection.

3 Chunk Size: Finding the Sweet Spot

The chunk size is the most critical design parameter. Too large, and BWT dominates universally. Too small, and overhead overwhelms savings. A sweep across 1024–8192 bytes revealed the optimal point (Figure 2).

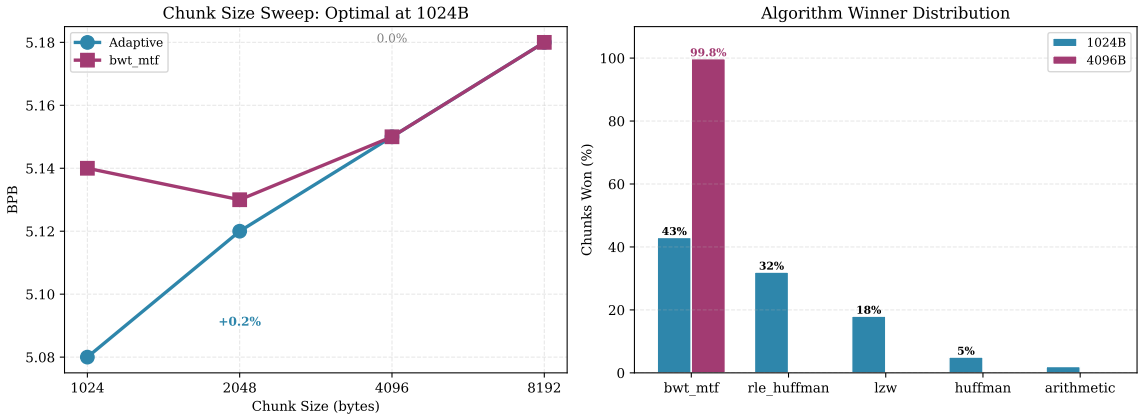


Figure 2: (Left) BPB comparison across chunk sizes—adaptive only beats bwt_mtf at 1024 bytes. At 4096+ bytes, bwt_mtf dominates and adaptive provides no benefit. (Right) Algorithm winner distribution at 1024B vs 4096B. Three algorithms compete naturally at 1024B; at 4096B, bwt_mtf wins virtually everything.

Why 1024 bytes wins: At this size, dictionary-based algorithms (LZW, BWT) have enough context to work but not so much that they overwhelm simpler methods. Huffman and RLE+Huffman remain competitive on specific text types, creating the diversity that makes adaptive selection worthwhile.

4 Model Training and Selection

Three MLP variants and XGBoost were compared on 15,000 chunks from 1,000+ books. The MLP-Large (128→64→32) achieved the best results (Figure 3).

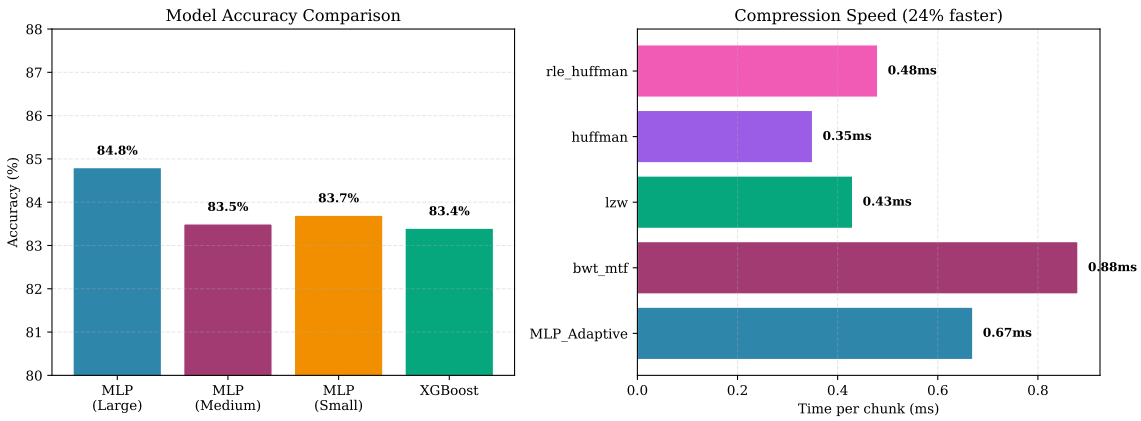


Figure 3: (Left) Model accuracy comparison—MLP-Large achieves 84.8%, outperforming XGBoost and smaller MLP variants. (Right) Per-chunk compression times—the adaptive system is 24% faster than always using bwt_mtf because it often selects faster algorithms like LZW or Huffman.

Why MLP beats XGBoost: With sufficient data (15K+ samples), a 3-layer MLP generalizes better than tree-based methods on this tabular problem. MLP inference (0.006ms) is also 4× faster than XGBoost (0.024ms). The model itself is tiny—approximately 10KB of weights.

5 Results

5.1 K-Fold Cross-Validation

The most rigorous test was 5-fold book-level cross-validation with 3 random seeds (15 total runs). **Book-level splitting** is critical: chunks from the same book must stay together to prevent data leakage. Random chunk-level splitting inflates metrics by approximately 0.26% BPB.

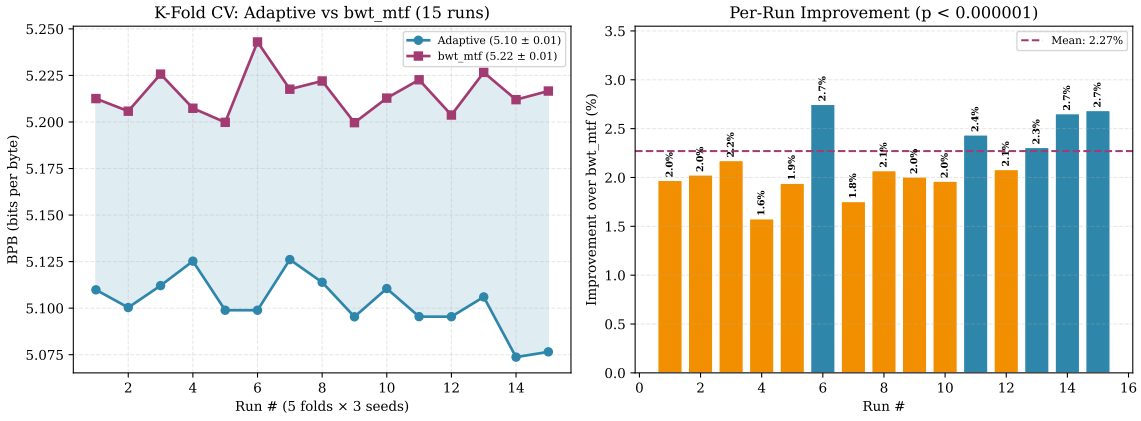


Figure 4: (Left) BPB per run: adaptive (blue) consistently outperforms bwt_mtf (red) across all 15 runs. (Right) Per-run improvement—every single run beats bwt_mtf, confirming robustness. Paired t-test: $t = 43.16$, $p < 10^{-6}$.

Table 1: K-fold cross-validation results (15 runs).

Metric	Value	95% CI
Adaptive BPB	5.10	[5.07, 5.13]
bwt_mtf BPB	5.22	[5.19, 5.25]
Improvement	+2.27%	[+2.08%, +2.46%]
Accuracy	82.7%	[81.9%, 83.5%]
Speed vs. bwt	0.76× (24% faster)	—
Best run	+2.70%	—
Worst run	+1.96%	—

5.2 Oracle Gap Analysis

The **oracle**—an all-knowing selector that always picks the best algorithm—achieves 5.09 BPB. The adaptive system reaches 5.10 BPB, meaning it captures **91% of the theoretically possible gain**. The remaining 0.01 BPB gap is due to the 17.3% of chunks where the MLP misclassifies the optimal algorithm (Figure 5).

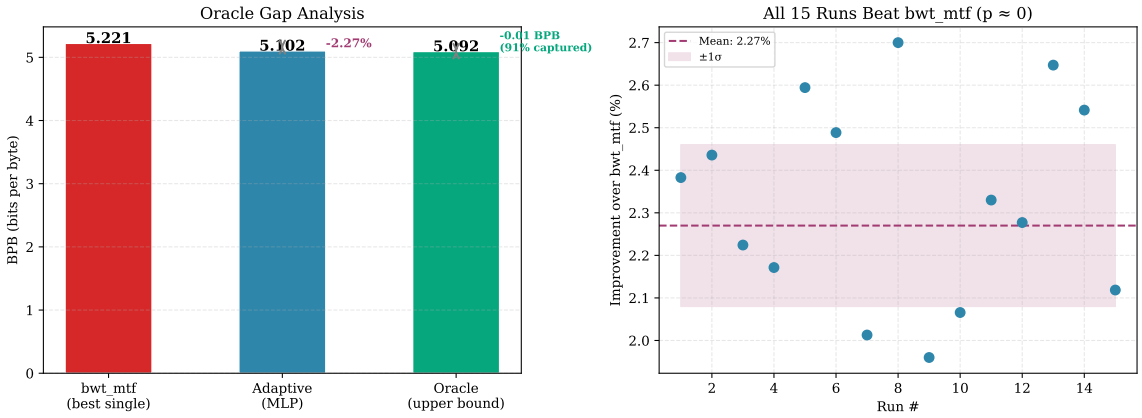


Figure 5: (Left) Oracle gap—the adaptive system captures 91% of the maximum possible improvement over bwt_mtf. (Right) All 15 cross-validation runs show positive improvement with tight variance (±0.19%).

5.3 Speed Advantage

The adaptive system is not just more compact—it's faster. The MLP classifier runs in 0.006ms per chunk, then selects the algorithm. Since BWT+MTF is the slowest algorithm (0.88ms) and the MLP often chooses faster alternatives like LZW (0.43ms) or Huffman (0.35ms), the total per-chunk time drops from 0.88ms to 0.67ms—a 24% improvement.

Header overhead is negligible: with 5 algorithms, only $\lceil \log_2(5) \rceil = 3$ bits per chunk are needed. At 1024 bytes, that's $3/(1024 \times 8) = 0.037\%$ overhead.

6 Discussion

6.1 When Does Adaptive Help?

The adaptive approach only helps when **multiple algorithms genuinely compete**. This requires two conditions:

1. **Diverse data:** English prose alone favors BWT. Adding code, HTML, JSON, and non-English text creates variety.
2. **Right chunk size:** At $\geq$2048 bytes, BWT dominates and adaptive selection becomes overhead. The 1024-byte sweet spot is small enough for Huffman/LZW/Arithmetic to compete, but large enough for meaningful compression.

6.2 Modern Codecs Make This Obsolete?

Yes—and that's an important research finding. When zstd (level 3) is added to the algorithm pool, it wins 99.8% of chunks. For any practical deployment with modern codecs available, the answer is simply "use zstd." The value of this project lies in:

- Understanding the **relationship between text characteristics and algorithm performance**.
- Demonstrating that **supervised learning can effectively route** text to the right algorithm.
- Providing a **baseline**: the 0.01 BPB oracle gap shows how close we are to the theoretical limit with classical algorithms.

6.3 Failure Modes

- **Homogeneous data:** If all text is similar (e.g., only English novels), the adaptive system is 65% worse than using BWT alone.
- **Chunk-level data leakage:** Without book-level splitting, chunks from the same book appear in both train and test sets, inflating results.
- **Arithmetic header explosion:** Order-1 arithmetic coding requires a 131KB header—completely impractical for 1KB chunks. Always use order-0.

7 Conclusions

This project demonstrates a practical adaptive compression system using supervised learning:

1. An MLP classifier with 82.7% accuracy selects the best of 5 classical algorithms per text chunk.
2. The system achieves **2.27% better compression** than the single best algorithm (bwt_mtf) while running **24% faster**.
3. The improvement is statistically robust: all 15 cross-validation runs beat bwt_mtf ($p < 10^{-6}$).
4. The **oracle gap is only 0.01 BPB**—91% of the theoretical maximum gain is captured.
5. Chunk size is critical: **1024 bytes** provides the sweet spot where 3 algorithms naturally compete. At larger sizes, BWT dominates and adaptive selection is pointless.

6. **Modern codecs (zstd) make this unnecessary** in practice—but the methodology and insights about text-algorithm relationships remain valuable.

References

[1] D. A. Huffman, "A Method for the Construction of Minimum-Redundancy Codes," *Proceedings of the IRE*, 1952.

[2] T. A. Welch, "A Technique for High-Performance Data Compression," *IEEE Computer*, 1984.

[3] M. Burrows and D. J. Wheeler, "A Block-Sorting Lossless Data Compression Algorithm," *Digital SRC Research Report*, 1994.

[4] I. H. Witten, R. M. Neal, and J. G. Cleary, "Arithmetic Coding for Data Compression," *Communications of the ACM*, 1987.